

Chaque commande est une structure, qui comporte les info nécessaire comme un tableau a mettre dans `execvp`, le nombre d'arguments de ce tableau , si la commande doit être faite dans le background, et enfin les files descriptors pour remplacer les `stdout` et `stdin` si écriture dans un fichier ou encore si utilisation de pipes.

On a aussi une structure de liste chaînées de commandes pour tout avoir au même endroit.

Et une structure globale pour tenir des truc important, comme la liste de commandes, ce que le user écrit etc...

Fonctionnement :

L'utilisateur rendre une commande exemple :

« `ls ; echo "a"` »

Le programme découpe en utilisant `strtok_k` d'abord les `;` puis les `|` ce qui nous donne les commandes séparée. C'est la qu'on affecte les pipes au `fd_in` et `fd_out` de nos commandes.

On détecte si il y a `>` pour une redirection on découpe les espaces puis on détecte la présence de l'esperluette pour les process dans le background.

On execute les dites commandes.

Les builtin et les commandes «normale » qui font appel a au `syscall execvp`.

Si il y a un `fd_out` ou un `fd_in` non zéro on utilise `dup2` pour remplacer `stdout` ou `stdin` par `fd_out` ou `fd_in` .

Dans le cas d'un process dans le background on ne met pas de `wait` sur ce `pid`.

Lorsqu'un background process ce fini un `child_handler` le détecte et on fait le `wait pid` a ce moment.

Les fichiers :

`Input.c` : prend en charge les input et le parsing.

`Utilities.c` : prend en charge les fonction utiles par exemple de nettoyage.

`Function.c` : l'execution des commandes.

`Main.c` : la boucle principale + le `child_handler`.